

Министерство науки и высшего образования Российской Федерации ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

ITMO University

Лабораторная работа 1

По дисциплине Администрирование платформ на ОС Linux

Тема работы Сертификаты

Обучающийся Рекун Ирина Данииловна

Факультет прикладной информатики

Группа K3220

Направление подготовки 11.03.02 Инфокоммуникационные технологии и системы связи

Образовательная программа Программирование в инфокоммуникационных системах

Обучающийся

(дата)

(подпись)

Рекун И.Д.

(Ф.И.О.)

Руководитель

(дата)

(подпись)

Береснев А.Д.

(Ф.И.О.)

СОДЕРЖАНИЕ

ВВЕДЕНИЕ.....	3
1. Создание корневого центра сертификации (Root CA)	4
2. Импорт корневого сертификата	5
3 Создание ключа для веб-сервера.	5
4. Создание запроса на подпись сертификата (CSR) с использованием ключа	5
5. Запуск приложения.....	6
Заключение.....	9

ВВЕДЕНИЕ

Цель работы: получить опыт выпуска и подписи самоподписанных сертификатов и использование доменного имени.

Задачи: в ходе выполнения лабораторной работы необходимо:

1. Сгенерировать корневой сертификат и приватный ключ центра сертификации (Root CA).
2. Импортировать созданный корневой сертификат в систему.
3. Создать RSA-ключ длиной 2048 бит для веб-сервера.
4. Сформировать запрос на подпись сертификата (CSR).
5. Передать CSR однопользователю и получить подписанный сертификат.
6. Импортировать корневой сертификат однопользователя и проверить полученный сертификат.
7. Создать простое HTTPS-приложение на Python.
8. Запустить сервер и проверить его работу с помощью команды curl.

1. Создание корневого центра сертификации (Root CA)

На первом этапе был создан корневой сертификат и приватный ключ центра сертификации. Ключ был создан с использованием алгоритма RSA длиной 2048 бит, а для подписи использовался алгоритм хеширования SHA-256. Срок действия сертификата был установлен на 10 лет. Во время выполнения команды необходимо было ввести информацию, которая будет включена в сертификат: страна, регион, организация и т. д. Результат выполнения команды представлен на рисунке 1.

[illegible]

Рисунок 1 – Генерация корневого сертификата и ключа

После выполнения команды были созданы два файла (Рисунок 2):

- ❖ root.key - приватный ключ центра сертификации
- ❖ root.crt - корневой сертификат

```
root@rekunid:~# ls
root.crt  root.key
root@rekunid:~#
```

Рисунок 2 – Созданные файлы root.key и root.crt

2. Импорт корневого сертификата

Далее импортируем корневой сертификат в систему для добавления в список доверенных сертификатов. Для этого сертификат копируем в каталог системных сертификатов и обновляем список сертификатов. В результате система добавила созданный сертификат в список доверенных. Процесс импорта сертификата изображен на рисунке 3.

```
root@rekunid:~# cp root.crt /usr/local/share/ca-certificates/root.crt
root@rekunid:~# update-ca-certificates
Updating certificates in /etc/ssl/certs...
rehash: warning: skipping ca-certificates.crt, it does not contain exactly one ce
rtificate or CRL
1 added, 0 removed; done.
Running hooks in /etc/ca-certificates/update.d...
done.
root@rekunid:~#
```

Рисунок 3 – Импорт корневого сертификата в систему

3 Создание ключа для веб-сервера.

Был создан RSA ключ для будущего веб-сервера. В результате выполнения команды был создан файл server.key, содержащий приватный ключ сервера. После генерации ключа была выполнена команда просмотра файлов каталога, которая подтвердила создание ключа сервера. Результат выполнения команды показан на рисунке 4

```
done.
root@rekunid:~# openssl genrsa -out server.key 2048
root@rekunid:~# ls
root.crt  root.key  server.key
root@rekunid:~#
```

Рисунок 4 – Генерация RSA ключа для веб-сервера

4. Создание запроса на подпись сертификата (CSR) с использованием ключа

На данном этапе был создан запрос на подпись сертификата CSR с использованием ранее созданного ключа сервера. В результате выполнения команды был создан файл server.csr, содержащий публичный ключ и информацию о владельце сертификата. Файл CSR используется для передачи в центр сертификации, который подписывает сертификат. Результат создания CSR показан на рисунке 5.

Файл CSR используется для передачи в центр сертификации, который подписывает сертификат. Результат создания CSR показан на рисунке 5.

```
root@rekunid:~# openssl req -new -key server.key -out server.csr -subj "/CN=garr  
iet.duckdns.org"  
root@rekunid:~# ls  
root.crt root.key server.csr server.key  
root@rekunid:~#
```

Рисунок 5 - Создание запроса на подпись сертификата

После создания файла запроса на подпись сертификата server.csr он был передан однокласснику для подписания. Передача файла выполнялась с использованием сетевой утилиты netcat (Рисунок 6).

```
root@rekunid:~# nc -w 3 178.72.165.136 7777 < server.csr  
root@rekunid:~# nc -l 7778 > my_signed.crt  
root@rekunid:~# openssl verify -CAfile friend_rootCA.crt my_signed.crt
```

Рисунок 6 - Передача CSR

Далее корневой сертификат одноклассника friend_rootCA.crt был импортирован в систему для добавления его в список доверенных сертификатов (Рисунок 7). Для этого была выполнена команда копирования сертификата в каталог системных сертификатов. В результате корневой сертификат одноклассника был добавлен в систему и может использоваться для проверки подписанных сертификатов.

```
root@rekunid:~# nc -l 7779 > friend_rootCA.crt  
root@rekunid:~# openssl verify -CAfile friend_rootCA.crt my_signed.crt  
my_signed.crt: OK  
root@rekunid:~# cp friend_rootCA.crt /usr/local/share/ca-certificates/friend_roo  
tCA.crt  
root@rekunid:~# update-ca-certificates  
Updating certificates in /etc/ssl/certs...  
rehash: warning: skipping ca-certificates.crt, it does not contain exactly one ce  
rtificate or CRL  
1 added, 0 removed; done.  
Running hooks in /etc/ca-certificates/update.d...  
done.  
root@rekunid:~#
```

Рисунок 7 – Получение подписанного сертификата и проверка сертификата

5. Запуск приложения

После получения подписанного сертификата был создан простой HTTPS-сервер на языке программирования Python. Для этого был создан файл app.py, содержащий код веб-сервера.

В программе используются следующие модули Python:

- ❖ http.server – для реализации HTTP-сервера
- ❖ socketserver – для работы с сетевыми соединениями
- ❖ ssl – для обеспечения защищённого TLS-соединения

В коде программы были указаны имя и фамилия студента, а также файлы сертификата и приватного ключа, используемые сервером:

- ❖ my_signed.crt – подписанный сертификат сервера
- ❖ server.key – приватный ключ сервера

Сервер был настроен на работу на порту 443, который используется для защищённых HTTPS-соединений. При обращении клиента к серверу программа обрабатывает HTTP-запрос и отправляет ответ, содержащий информацию о виртуальной машине студента и IP-адресе клиента. Фрагмент программы приведён на рисунке 8.

```
import http.server
import socketserver
import ssl
import os

STUDENT_SURNAME = "Rekun"
STUDENT_NAME = "Irina"
CERT_FILE = "my_signed.crt"
KEY_FILE = "server.key"
PORT = 443

class MyHandler(http.server.SimpleHTTPRequestHandler):
    def do_GET(self):
        self.send_response(200)
        self.send_header("Content-type", "text/plain; charset=utf-8")
        self.end_headers()
        response_text = (
            f"This is a student's virtual machine: {STUDENT_SURNAME} {STUDENT_NAME}\n"
            f"IP-adres: {self.client_address[0]}\n"
        )

        self.wfile.write(response_text.encode("utf-8"))
httpd = socketserver.TCPServer(("", PORT), MyHandler)
context = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)
context.load_cert_chain(certfile=CERT_FILE, keyfile=KEY_FILE)
```

Рисунок 8 – Код программы HTTPS-сервера

После запуска программы в терминале появилось сообщение, представленное на рисунке 9. Так, сервер успешно запущен и готов принимать защищённые HTTPS-соединения от клиентов.

```
root@rekunid:~# sudo python3 app.py
HTTPS server is running on port 443
```

Рисунок 9 – Запуск HTTPS-сервера

Для проверки работы созданного HTTPS-сервера используем утилиту curl, позволяющая выполнять HTTP-запросы из командной строки. Параметр -v позволяет вывести подробную информацию о процессе соединения, включая этапы TLS-рукопожатия и обмена HTTP-запросами. Параметр -k используется для игнорирования предупреждения о недоверенном сертификате, так как в лабораторной работе используется самоподписанный сертификат. В результате выполнения команды сервер успешно обработал запрос и вернул HTTP-ответ с кодом 200, что означает успешную обработку запроса, также в выводе команды отображается строка: GET / HTTP/1.1 200, которая подтверждает получение и обработку HTTP-запроса сервером.

```
root@rekunid:~# sudo python3 app.py
HTTPS server is running on port 443
curl -v -k https://178.72.166.217
5.255.231.72 - - [06/Mar/2026 00:47:27] "GET / HTTP/1.1" 200 -
```

Рисунок 9 — Проверка работы HTTPS-сервера с помощью curl

Веб-сервер также был проверен через браузер. Для этого в адресной строке браузера необходимо было написать IP-адрес виртуальной машины. После открытия страницы браузер отобразил предупреждение о небезопасном соединении, так как используется самоподписанный сертификат, который не входит в список доверенных центров сертификации. После игнорирования предупреждения открылась страница, содержащая ответ сервера. На странице отобразилось сообщение с информацией о виртуальной машине и IP-адресе клиента, который выполнил запрос к серверу (Рисунок 10).

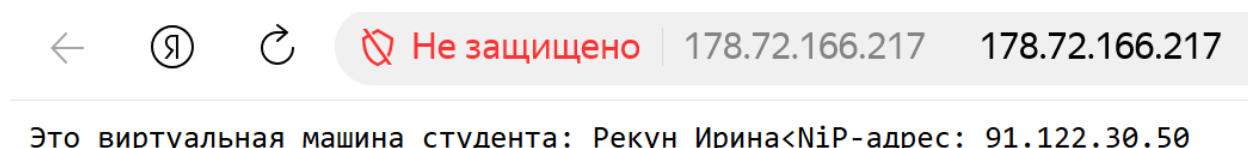


Рисунок 10 — Проверка работы HTTPS-сервера в браузере

Заключение

В ходе выполнения лабораторной работы был создан корневой центр сертификации, сгенерированы ключи и сформирован запрос на подпись сертификата (CSR). Полученный запрос был передан одногруппнику и подписан его корневым сертификатом. После этого была выполнена проверка полученного сертификата и его импорт в систему доверенных сертификатов. Также был разработан и запущен простой HTTPS-сервер на языке Python, использующий подписанный сертификат и приватный ключ сервера. Работа сервера была успешно проверена с помощью браузера и утилиты curl. В результате выполнения лабораторной работы были получены практические навыки работы с цифровыми сертификатами.

Ответы на вопросы

1. Почему было предупреждение в браузере при открытии домена?

Предупреждение появилось потому, что используется само подписанный сертификат. Такой сертификат не выдан официальным центром сертификации, поэтому браузер не может автоматически доверять ему. Из-за этого браузер предупреждает пользователя о возможной небезопасности соединения.

2. Что могло бы произойти, если бы Вы одnogруппнику передавали ключ?

Если передать приватный ключ одnogруппнику, это может привести к проблемам с безопасностью. Приватный ключ должен храниться только у владельца, так как с его помощью можно расшифровывать данные или выдавать себя за владельца сервера. Если ключ попадёт к другому человеку, он сможет использовать его для подделки сертификатов или перехвата защищённого соединения.

3. В общих чертах, как происходит шифровка данных с помощью сертификата?

Когда клиент подключается к серверу, сервер отправляет свой сертификат, в котором содержится публичный ключ. Клиент проверяет этот сертификат и использует публичный ключ для начала защищённого соединения. После этого стороны договариваются о симметричном ключе, который используется для дальнейшего шифрования данных, так как он работает быстрее.

4. Почему система доверяет сертификатам, выписанным коммерческими организациями?

Браузеры и операционные системы имеют встроенный список доверенных центров сертификации. Коммерческие центры сертификации проходят проверку и добавляются в этот список. Поэтому сертификаты, подписанные такими организациями, автоматически считаются доверенными и браузер не показывает предупреждение.